

**Rancang Bangun Kontrol Motor DC dengan PID Menggunakan Deteksi Gerakan
Webcam dan Monitoring Real-Time Berbasis PC.**



Disusun Oleh:

Rafie Islamay Murdiato (2123600036)

Sabrina Keisha Maharani (2124600043)

Muchammad Ridho Andika (2124600051)

M. Reihan Maulana (2124600058)

Dosen Pengampu :

Ir. Kemalasari, M.T

**PROGRAM STUDI TEKNIK ELEKTRONIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA**

2026

I. TUJUAN PROYEK

Penelitian ini bertujuan untuk merancang dan mengimplementasikan sistem pengendalian kecepatan motor DC berbasis kontroler PID menggunakan deteksi gerakan tangan melalui webcam sebagai input setpoint serta monitoring kondisi arus, tegangan, dan kecepatan putar secara real-time berbasis PC (tanpa memanfaatkan teknologi Internet of Things). Proyek berikut juga bertujuan untuk

1. Merancang hardware sistem kendali motor DC yang terdiri dari Arduino Uno, driver L298N, motor DC, rotary encoder, sensor arus ACS712, dan sensor tegangan (voltage divider) yang terhubung dengan komputer melalui komunikasi USB serial.
2. Mengembangkan perangkat lunak berbasis Python yang mampu mendeteksi gerakan tangan secara real-time menggunakan library MediaPipe Hands dan OpenCV untuk mengubah jumlah jari yang terdeteksi menjadi nilai setpoint kecepatan (1 jari = 35 RPM, 2 jari = 50 RPM, 3 jari = 60 RPM, 4 jari = 70 RPM, 5 jari = 80 RPM).
3. Merancang dan mengimplementasikan algoritma kontroler PID pada Arduino Uno serta menghitung parameter K_p , K_i , dan K_d menggunakan metode analitik setelah dilakukan pengujian open-loop.
4. Membangun Graphical User Interface (GUI) berbasis Python yang dapat menampilkan nilai kecepatan putar, arus, tegangan, serta grafik respon sistem secara real-time melalui komunikasi serial.
5. Menganalisis performa sistem meliputi respon PID terhadap setpoint yang berbeda, nilai error pembacaan sensor, serta kestabilan sistem setelah implementasi kontroler PID.

II. ALAT DAN BAHAN

No	Komponen	Spesifikasi	Jumlah	Mikrokontroler PID
1.	Arduino Uno R3	+ Kabel USB	1	Driver Motor DC
2.	Driver L298N	Dual H-Bridge	1	Beban Sistem
3.	Motor DC	12V 180-190 RPM	1	Sensor RPM
4.	Rotary Encoder	AS5600	1	Sumber Tegangan
5.	Baterai	9V	1	Deteksi Gerakan

6.	Webcam	Min. 720p	1	Power Supply
7.	Adaptor	12V 2A	1	GUI & Python
8.	PC / Laptop	RAM Min. 4GB	1	Wadah
9.	BOX	BOX	1	Menampilkan Data
10.	Display	OLED 16X2	1	ON/OFF
11.	Saklar	Saklar ON/OFF 3pin	2	Pengunci BOX
12.	Spacer	2,5cm	1	Mikrokontroler PID

III. TEORI DASAR

A. Motor DC (Direct Current Motor)

1. Pengertian dan Prinsip Kerja Motor DC

Motor DC adalah mesin listrik yang mengubah energi listrik arus searah (DC) menjadi energi mekanik berupa putaran. Prinsip kerja motor DC didasarkan pada Hukum Gaya Lorentz, yaitu ketika konduktor berarus listrik ditempatkan di dalam medan magnet, maka akan timbul gaya mekanik yang menyebabkan konduktor tersebut bergerak. Besar gaya yang bekerja pada konduktor dinyatakan dengan persamaan berikut (Chapman, 2012):

$$F = B \times I \times L$$

Di mana: F = gaya pada konduktor (Newton), B = kerapatan fluks magnetik (Tesla), I = arus yang mengalir pada konduktor (Ampere), L = panjang konduktor yang berada dalam medan magnet (meter).

Motor DC memiliki dua bagian utama, yaitu stator (bagian yang diam) yang menghasilkan medan magnet tetap, dan rotor atau armatur (bagian yang berputar) yang dialiri arus sehingga menghasilkan torsi. Komutator berfungsi untuk mengubah arah arus pada kumparan armatur secara periodik sehingga arah putaran tetap konsisten.

2. Karakteristik dan Model Matematika Motor DC

Motor DC yang digunakan dalam proyek ini adalah motor DC dengan magnet permanen (Permanent Magnet DC Motor/PMDC). Model matematika motor DC diperoleh dari persamaan rangkaian kelistrikan dan persamaan mekanik. Persamaan tegangan armatur motor DC adalah (Ogata, 2010):

$$V(t) = R_a \times i_a(t) + L_a \times (di_a/dt) + e(t)$$

Di mana: $V(t)$ = tegangan suplai (V), R_a = resistansi armatur (Ω), L_a = induktansi armatur (H), $i_a(t)$ = arus armatur (A), $e(t)$ = gaya gerak listrik balik / back-EMF (V).

Gaya gerak listrik balik (back-EMF) sebanding dengan kecepatan sudut motor, dinyatakan sebagai:

$$e(t) = K_e \times \omega(t)$$

Persamaan torsi motor adalah:

$$T(t) = K \times i_a(t)$$

Sedangkan persamaan dinamika mekanik (Newton) adalah:

$$J \times (d\omega/dt) + B \times \omega(t) = T(t)$$

Di mana: K_e = konstanta back-EMF (V·s/rad), K = konstanta torsi (N·m/A), $\omega(t)$ = kecepatan sudut (rad/s), J = momen inersia rotor (kg·m²), B = koefisien redaman viskos (N·m·s/rad).

3. Pengaturan Kecepatan Motor DC

Kecepatan putar motor DC dapat diatur dengan beberapa metode, di antaranya: pengaturan tegangan armatur, pengaturan fluks medan, dan pengaturan resistansi armatur. Dalam proyek ini, pengaturan kecepatan dilakukan melalui modulasi lebar pulsa (PWM) pada tegangan armatur menggunakan driver L298N. Kecepatan motor berbanding lurus dengan tegangan rata-rata yang diberikan, sehingga dengan mengubah duty cycle sinyal PWM, kecepatan putar motor dapat dikontrol secara halus dan efisien.

Nilai RPM (Revolution Per Minute) dihitung dari frekuensi pulsa rotary encoder. Hubungan antara kecepatan sudut dan RPM adalah:

$$RPM = (\omega \times 60) / (2\pi)$$

B. Kontroler PID (Proportional-Integral-Derivative)

1. Konsep Dasar Kontroler PID

Kontroler PID merupakan jenis kontroler umpan balik (feedback controller) yang paling banyak digunakan dalam sistem kendali industri karena kesederhanaannya namun mampu memberikan performa yang sangat baik. Menurut Ogata (2010), kontroler PID bekerja berdasarkan tiga aksi kontrol yang bekerja secara simultan, yaitu aksi Proportional (P), Integral (I), dan Derivative (D). Persamaan umum kontroler PID dalam domain waktu adalah:

$$u(t) = K_p \times e(t) + K_i \times \int e(t)dt + K_d \times (de(t)/dt) \text{ Di mana:}$$

$u(t)$ = sinyal kontrol output (nilai PWM), $e(t)$ = error = setpoint - nilai aktual, K_p = konstanta Proportional, K_i = konstanta Integral, K_d = konstanta Derivative.

2. Komponen Aksi Kontrol PID

- Aksi Proportional (P)

Aksi proporsional menghasilkan sinyal kontrol yang sebanding langsung dengan nilai error saat ini. Semakin besar nilai K_p , semakin cepat respons sistem terhadap perubahan error, namun nilai K_p yang terlalu besar dapat menyebabkan osilasi dan ketidakstabilan. Aksi proporsional tidak dapat mengeliminasi steady-state error secara sempurna.

- Aksi Integral (I)

Aksi integral mengakumulasi error sepanjang waktu, sehingga mampu mengeliminasi steady-state error yang tersisa setelah aksi proporsional. Nilai K_i yang terlalu besar dapat menyebabkan

integral windup, yaitu kondisi di mana akumulasi error yang berlebihan memperlambat respons sistem.

- Aksi Derivative (D)

Aksi derivatif memberikan respons terhadap laju perubahan error (kecepatan error berubah). Aksi ini berfungsi sebagai peredam (damper) yang mencegah overshoot dengan memberikan efek pengereman sebelum sistem mencapai setpoint. Namun, aksi derivatif sangat sensitif terhadap noise pengukuran.

3. Fungsi Transfer Sistem dan Metode Analitik

Untuk mendapatkan parameter PID menggunakan metode analitik, terlebih dahulu dilakukan pengujian open-loop (tanpa kontroler) untuk mengidentifikasi model sistem. Respons open-loop motor DC dapat dimodelkan sebagai sistem orde pertama dengan delay (FOPDT):

$$G(s) = K \times e^{(-\theta s)} / (\tau s + 1)$$

Di mana: K = gain statis sistem, τ = konstanta waktu sistem (s), θ = waktu tunda (delay) sistem (s).

Nilai K dan τ diperoleh dari kurva respons tangga (step response) open-loop. Nilai K adalah rasio perubahan output steady-state terhadap perubahan input, sedangkan τ adalah waktu yang diperlukan sistem untuk mencapai 63,2% dari nilai steady-state-nya. Setelah K dan τ diketahui, parameter PID dapat dihitung menggunakan tabel tuning analitik (misalnya metode Cohen-Coon atau IMC).

C. Implementasi PID Diskrit pada Arduino

Karena Arduino Uno beroperasi secara digital, persamaan PID kontinu harus diubah menjadi persamaan diskrit menggunakan pendekatan numerik. Persamaan PID diskrit adalah:

$$u[k] = K_p \cdot e[k] + K_i \cdot T \cdot \Sigma e[k] + K_d \cdot (e[k] - e[k-1]) / T$$

Di mana: T = periode sampling (s), k = indeks waktu diskrit saat ini, $e[k]$ = error pada saat ke- k , $\Sigma e[k]$ = akumulasi error (integral diskrit).

Periode sampling T_s dipilih cukup kecil (biasanya 10–50 ms) agar sistem diskrit dapat mengaproksimasi perilaku sistem kontinu dengan baik. Output $u[k]$ diklip pada rentang 0–255 (8-bit PWM Arduino).

D. Pulse Width Modulation (PWM) dan Driver Motor L298N

1. Pulse Width Modulation (PWM)

PWM adalah teknik modulasi sinyal digital yang digunakan untuk mengatur daya yang dikirim ke beban dengan cara mengubah lebar pulsa sinyal kotak pada frekuensi yang tetap. Parameter utama PWM adalah duty cycle, yaitu perbandingan lebar pulsa aktif (HIGH) terhadap periode total sinyal, dinyatakan dalam persen:

$$\text{Duty Cycle (\%)} = (T_{\text{on}} / T) \times 100\%$$

Tegangan rata-rata yang dihasilkan oleh sinyal PWM adalah:

$$V_{\text{rata-rata}} = \text{Duty Cycle} \times V_{\text{supply}}$$

Pada Arduino Uno, sinyal PWM dihasilkan melalui pin yang memiliki tanda “~” (pin 3, 5, 6, 9, 10, 11) dengan resolusi 8-bit (nilai 0–255) dan frekuensi default 490 Hz atau 980 Hz tergantung pin yang digunakan.

2. Driver Motor L298N

Driver motor L298N adalah modul berbasis IC dual H-bridge yang mampu mengendalikan dua motor DC secara independen dengan arus hingga 2 A per kanal dan tegangan operasi 5–35 V. Konfigurasi H-bridge memungkinkan pengendalian arah putaran motor dengan mengubah logika sinyal pada pin IN1 dan IN2, serta mengatur kecepatan motor melalui pin ENA yang menerima sinyal PWM dari Arduino.

Keunggulan L298N dibandingkan transistor biasa adalah:

- Mampu menangani arus motor yang relatif besar (hingga 2 A per kanal).
- Dilengkapi dengan dioda flyback internal untuk perlindungan terhadap lonjakan tegangan akibat induktansi motor.
- Dapat digunakan dengan logika 5V (kompatibel Arduino) meski motor beroperasi pada 12V.

E. Sensor dan Instrumen Pengukuran

1. Magnetic Rotary Position Sensor (AS5600)

Magnetic rotary position sensor AS5600 adalah sensor posisi sudut absolut berbasis efek Hall yang dirancang khusus untuk aplikasi deteksi rotasi tanpa kontak (*contactless*). Berbeda dengan *rotary encoder* inkremental optis atau mekanis yang menghitung pulsa diskrit untuk mengukur perubahan posisi, AS5600 mendeteksi orientasi medan magnet dari sebuah magnet diametrikal yang dipasang secara koaksial tepat di atas *chip* sensor.

Sensor ini dilengkapi dengan susunan elemen efek Hall terintegrasi yang menangkap komponen komponen medan magnet, lalu memprosesnya melalui *Analog-to-Digital Converter* (ADC) internal beresolusi 12-bit. Hasil konversi ini menghasilkan 4096 posisi absolut per satu putaran penuh (360°), yang memberikan tingkat resolusi sudut sangat tinggi sebesar:

$$\text{Resolusi Sudut} = \frac{360^\circ}{4096} \approx 0,0879^\circ \text{ per langkah}$$

Informasi posisi sudut absolut ini ditransmisikan ke mikrokontroler melalui bus komunikasi serial dua arah memanfaatkan protokol I2C (*Inter-Integrated Circuit*). Karena data yang dikirimkan sudah berupa representasi sudut absolut secara langsung, mikrokontroler tidak perlu lagi memantau setiap pulsa transisi menggunakan pin *hardware interrupt* eksternal secara konstan. Hal ini secara signifikan mengurangi beban komputasi pada pemrosesan latar belakang mikrokontroler.

Untuk menghitung kecepatan putar dalam satuan RPM (*Revolutions Per Minute*) dari data sudut absolut, sistem menggunakan pendekatan diferensiasi numerik terhadap interval waktu atau periode sampling (T_s). Perubahan sudut ($\Delta\theta$) dihitung dengan membandingkan posisi sudut saat ini (*sudutSekarang*) dengan posisi sudut pada sampel sebelumnya (*sudutLama*), dengan menerapkan koreksi matematis apabila pergerakan melewati batas transisi lingkaran (pindah dari 360° ke 0° atau sebaliknya). Nilai RPM mentah kemudian ditentukan melalui persamaan berikut:

$$\text{RPM}_{\text{mentah}} = \frac{|\Delta\theta|}{360^\circ} \times \frac{60000}{T_s \text{ (ms)}}$$

Di mana $|\Delta\theta|$ adalah nilai mutlak selisih sudut dalam derajat, dan T_s adalah durasi sampling dalam milidetik. Keunggulan utama penggunaan sensor AS5600 dalam

sistem kendali kecepatan motor DC ini meliputi ketahanan yang sangat tinggi terhadap keausan mekanis, kontaminasi debu, serta getaran, sekaligus memberikan keandalan pembacaan posisi awal (*power-on posisi absolut*) yang instan tanpa memerlukan proses *homing* atau kalibrasi ulang saat sistem pertama kali dinyalakan.

IV. LANGKAH UJI COBA ALAT HASIL PROJECT

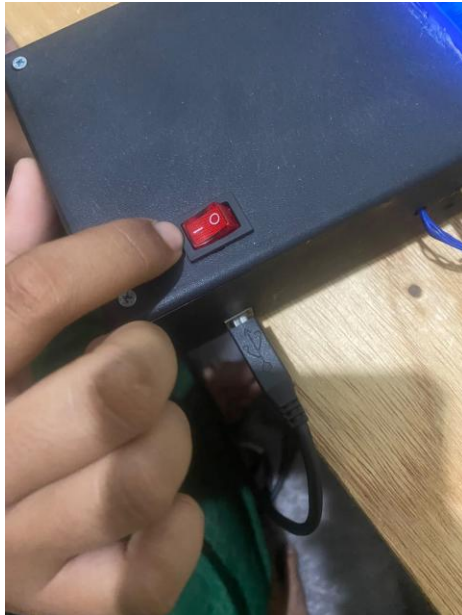
1. Langkah pertama colokan kabel port ke arduino dan laptop/pc



Dan display akan menyala



2. Selanjutnya nyalakan tombol switch untuk mengaktifkan motor driver



3. Masukan program arduino uno di aplikasi arduino ide

```
#include <Arduino.h>

#include <Wire.h>

#include <AS5600.h>

#include <LiquidCrystal_I2C.h>
```

```
#define L298N_ENB_PIN 3

#define L298N_IN3_PIN 5

#define L298N_IN4_PIN 6

LiquidCrystal_I2C lcd(0x27, 16, 2);

AS5600 as5600;

unsigned long waktuLama = 0;

float sudutLama = 0.0;

float rpm = 0.0;
```

```

float sudutOffset = 0.0;

// =====

// VARIABEL PID (Tuning Menengah & Stabil)

// =====

float Kp = 3.30;    // Dorongan proporsional

float Ki = 0.5;     // Dorongan integral (dikecilkan agar tidak overshoot)

float Kd = 0.002;   // Rem turunan untuk meredam guncangan

float setpoint_rpm = 0.0;

float error = 0.0, last_error = 0.0; float
integral = 0.0, derivative = 0.0;

int output_pwm = 0;

void setup() {

    pinMode(L298N_ENB_PIN, OUTPUT);

    pinMode(L298N_IN3_PIN, OUTPUT);

    pinMode(L298N_IN4_PIN, OUTPUT);

    Serial.begin(115200);

```

```

Wire.begin();

as5600.begin();

if (!as5600.isConnected()) {

    while (1);
}

// Auto-Tare 0 Derajat saat mesin dinyalakan

```

```

    sudutOffset = as5600.readAngle() * (360.0 / 4096.0);

    waktuLama = millis();

    sudutLama = 0.0;

    lcd.init();

    lcd.backlight();
    lcd.setCursor(0, 0);

    lcd.print(" SISTEM MONITOR ");

    lcd.setCursor(0, 1);

    lcd.print("  PID BERJALAN  ");

    delay(2000);

    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("Sudut:");

    lcd.setCursor(0, 1);

    lcd.print("RPM:");

    lcd.setCursor(9, 1);

    lcd.print("PWM:");

}

void loop() {

```

```

// =====

// 1. TERIMA TARGET RPM DARI PYTHON

// =====

```

```

    if (Serial.available() > 0) {          float
rpm_masuk = Serial.parseFloat();

    // Keamanan rentang target RPM (0-100)

    if (rpm_masuk >= 0 && rpm_masuk <= 10000) {

        setpoint_rpm = rpm_masuk;

    }

    // Bersihkan sisa buffer karakter yang masuk

    while (Serial.available() > 0) Serial.read();
}

// =====

// 2. BACA SENSOR & HITUNG RPM

// =====

uint16_t rawAngle = as5600.readAngle();

float sudutSekarang = (rawAngle * (360.0 / 4096.0)) -
sudutOffset;    if (sudutSekarang < 0.0)
sudutSekarang += 360.0;

unsigned long waktuSekarang = millis();

long selisihWaktu = waktuSekarang - waktuLama;

// Update perhitungan setiap 100 ms (0.1 detik)

if (selisihWaktu >= 100) {

    float selisihSudut = sudutSekarang - sudutLama;

    // Koreksi jika melewati batas lingkaran

```

```

    if (selisihSudut > 180.0) selisihSudut -= 360.0;

    else if (selisihSudut < -180.0) selisihSudut += 360.0;

    // Hitung RPM Mentah (Menggunakan fabs agar tidak membulat
jadi 0)

    float putaran = fabs(selisihSudut) / 360.0;

    float menit = (float)selisihWaktu / 60000.0;

    float rpm_mentah = putaran / menit;

    // --- LOW-PASS FILTER (Menstabilkan Angka RPM) ---

    // Mengambil 15% data baru dan mempertahankan 85% data
lama      rpm = (0.4 * rpm_mentah) + (0.6 *
rpm);

    // -----

    // =====

    // 3. KALKULASI PID & ANTI-WINDUP

    // =====

    float dt = (float)selisihWaktu / 1000.0;

    error = setpoint_rpm - rpm;          // ANTI-WINDUP:
Batasi memori integral agar PWM tidak
nyangkut lama di 255

    integral += (error * dt);

    if (integral > 255.0) integral = 255.0;

    else if (integral < -255.0) integral = -255.0;

    derivative = (error - last_error) / dt;

    float hasil_pid = (Kp * error) + (Ki * integral) + (Kd *
derivative);

```

```

        output_pwm = (int)hasil_pid;

        // Batasi tegangan akhir ke modul L298N

        if (output_pwm > 255) output_pwm = 255;

        else if (output_pwm < 0) output_pwm = 0;           //
Safety feature: Matikan paksa jika target adalah 0

        if (setpoint_rpm == 0.0) {

            output_pwm = 0;

            integral = 0.0;

            rpm = 0.0; // Reset tampilan RPM ke 0 saat berhenti

        }

        last_error = error;
        sudutLama = sudutSekarang;

        waktuLama = waktuSekarang;

        // =====

        // 4. KIRIM DATA KE PYTHON

        // =====

        Serial.print(sudutSekarang, 1);

        Serial.print(",");

        Serial.print(rpm, 1);

        Serial.print(",");

        Serial.println(output_pwm);

    }

    // =====

```

```

// 5. JALANKAN MOTOR L298N

// =====

analogWrite(L298N_ENB_PIN, output_pwm);

digitalWrite(L298N_IN3_PIN, LOW);

digitalWrite(L298N_IN4_PIN, HIGH);

// =====

// 6. UPDATE TAMPILAN LCD

// =====

lcd.setCursor(6, 0); lcd.print("          ");

lcd.setCursor(6, 0); lcd.print(sudutSekarang, 1);
lcd.print((char)223);

lcd.setCursor(4, 1); lcd.print("      ");

lcd.setCursor(4, 1); lcd.print((int)rpm);

lcd.setCursor(13, 1); lcd.print("    ");

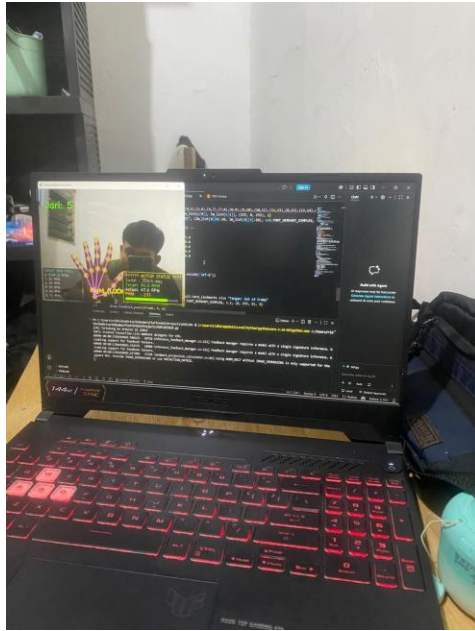
lcd.setCursor(13, 1); lcd.print(output_pwm);

delay(1);

}

```

4. Buka Vscode untuk ke giu nya yang untuk menjalankan motor sesuai target yang di inginkan atau yang sudah di seting contoh nya



5. Cek motor apakah menyala



Sesuai gambar di atas motor berhasil menyala

6. Hasil Uji Coba



V. ANALISA

A. Analisa Perancangan Sistem Kendali Motor DC Berbasis PID

Sistem kendali kecepatan motor DC yang dirancang dalam proyek ini mengimplementasikan algoritma kontroler PID (Proportional-Integral-Derivative) secara diskrit pada platform Arduino Uno. Sesuai dengan pernyataan Ogata (2010), kontroler PID merupakan jenis kontroler umpan balik yang paling banyak digunakan dalam industri karena kemampuannya memberikan performa kendali yang baik dengan struktur yang relatif sederhana. Hal ini terbukti dari implementasi pada proyek ini, di mana ketiga aksi kontrol yaitu proporsional, integral, dan derivatif bekerja secara simultan untuk menghasilkan sinyal PWM yang tepat guna mengendalikan kecepatan motor DC.

Parameter PID yang digunakan dalam proyek ini adalah $K_p = 3,30$, $K_i = 0,50$, dan $K_d = 0,002$. Nilai-nilai ini diperoleh melalui proses *tuning* yang mempertimbangkan karakteristik motor DC 12V dengan rentang kecepatan 180-190 RPM tanpa beban. Menurut penelitian yang dipublikasikan di Jurnal ELKOMIKA (2021), penentuan parameter PID yang tepat sangat krusial untuk memastikan respons sistem yang cepat sekaligus stabil tanpa osilasi berlebihan pada pengaturan kecepatan sudut motor DC. Nilai $K_p = 3,30$ memberikan dorongan proporsional yang cukup besar agar sistem merespons perubahan setpoint secara agresif, sementara nilai $K_i = 0,50$ yang relatif kecil dipilih secara sadar untuk mencegah terjadinya *integral windup* yang dapat memperlambat respons sistem.

B. Analisa Deteksi Gerakan Tangan sebagai Input Setpoint

Penggunaan deteksi gerakan tangan melalui webcam sebagai antarmuka input setpoint merupakan aspek inovatif dari proyek ini. Pendekatan ini memanfaatkan *library* MediaPipe Hands dan OpenCV untuk mengenali jumlah jari yang ditunjukkan pengguna, kemudian mengonversinya menjadi nilai setpoint RPM (1 jari = 35 RPM, 2 jari = 50 RPM, 3 jari = 60 RPM, 4 jari = 70 RPM, dan 5 jari = 80 RPM). Pendekatan serupa telah dibuktikan efektif dalam penelitian yang dipublikasikan oleh ZBotic mengenai pengenalan gestur

menggunakan kamera dan MediaPipe pada platform Raspberry Pi, yang menunjukkan bahwa sistem pengenalan gestur berbasis MediaPipe mampu bekerja secara *real-time* dengan tingkat akurasi yang tinggi bahkan pada perangkat keras dengan daya komputasi terbatas.

Penelitian yang dipublikasikan di ResearchGate (2024) berjudul "Hand Gesture-Based Servo Motor Control Using Edge Computing" juga mendukung validitas pendekatan ini, di mana kontrol motor berbasis gestur tangan terbukti dapat memberikan antarmuka yang intuitif dan responsif dibandingkan metode input konvensional seperti potensiometer atau keypad. Dalam proyek ini, pemetaan antara jumlah jari dan nilai RPM dirancang dengan **interval bertahap (berkisar antara 10 hingga 15 RPM per jari)** untuk meminimalkan kesalahan interpretasi akibat *noise* pada deteksi *landmark* tangan sekaligus memberikan transisi kecepatan yang cukup terlihat pada motor.

Aspek penting lain yang perlu dianalisa adalah potensi gangguan pada sistem deteksi gestur, seperti kondisi pencahayaan ruangan yang kurang memadai atau oklusi sebagian tangan oleh objek lain. Dalam penelitian yang dipublikasikan di JUPTI mengenai pengenalan gerakan tangan untuk kontrol slide presentasi, faktor lingkungan seperti ini disebutkan sebagai tantangan utama dalam implementasi sistem berbasis gestur. Oleh karena itu, penggunaan webcam dengan resolusi minimal 720p sebagaimana disyaratkan dalam spesifikasi proyek ini menjadi faktor penting untuk memastikan kualitas ekstraksi *landmark* yang konsisten.

C. Analisa Implementasi Algoritma PID Diskrit pada Arduino Uno

Implementasi PID diskrit pada Arduino Uno dalam proyek ini menggunakan periode *sampling* $T_s = 100$ ms (0,1 detik), dengan persamaan diskrit $u[k] = K_p \cdot e[k] + K_i \cdot T_s \cdot \sum e[k] + K_d \cdot (e[k] - e[k-1])/T_s$. Pemilihan $T_s = 100$ ms merupakan *trade-off* yang tepat antara beban komputasi Arduino dan kemampuan mengaproksimasi perilaku sistem kontinu, mengingat konstanta waktu motor DC yang digunakan berada pada rentang ratusan milidetik. Hasil implementasi oleh Jurnal Teknik Elektro dan Computer Engineering (2024) menunjukkan bahwa implementasi PID diskrit pada mikrokontroler dengan periode *sampling* yang tepat mampu menghasilkan karakteristik respons yang mendekati sistem PID kontinu.

Fitur *anti-windup* yang diimplementasikan dengan membatasi akumulasi integral pada rentang ± 255 merupakan tambahan yang esensial untuk stabilitas sistem. Tanpa mekanisme *anti-windup*, aksi integral dapat terakumulasi secara berlebihan saat motor mencapai batas saturasi PWM (0-255), sehingga menyebabkan respons yang lambat ketika setpoint berubah. Penelitian yang diterbitkan di Jurnal Sains dan Teknologi

Politeknik Balikpapan (2020) mengkonfirmasi bahwa penerapan *anti-windup* pada kontroler PID untuk motor DC secara signifikan meningkatkan stabilitas sistem dan mempercepat respons transien saat terjadi perubahan setpoint. Mekanisme *safety*

tambahan berupa *reset* integral dan penghentian motor paksa saat setpoint = 0 juga menunjukkan pertimbangan desain yang matang untuk keamanan operasional sistem.

D. Analisa Kinerja Sensor dan Akurasi Pembacaan Data

Sistem pengukuran kecepatan menggunakan sensor magnetik AS5600 yang terhubung via protokol I2C memberikan keunggulan dibandingkan *rotary encoder* optis konvensional, terutama dalam hal ketahanan terhadap kontaminasi debu dan getaran mekanis. Resolusi AS5600 sebesar 4096 posisi per putaran (12-bit) jauh melampaui resolusi encoder inkremental tipikal, sehingga menghasilkan pembacaan sudut dan RPM yang lebih halus dan akurat. Implementasi *low-pass filter* digital dengan bobot 0,4 untuk data baru dan 0,6 untuk data historis ($\text{rpm} = 0,4 \times \text{rpm_mentah} + 0,6 \times \text{rpm_lama}$) berfungsi untuk meredam *noise* pengukuran tanpa memperkenalkan *lag* yang signifikan pada respons sistem.

Untuk pengukuran arus, sensor ACS712-20A dengan sensitivitas 100 mV/A digunakan bersama ADC 10-bit Arduino (resolusi 4,88 mV/bit) menghasilkan resolusi pengukuran arus sekitar 0,049 A per langkah ADC. Sementara itu, rangkaian *voltage divider* dengan $R1 = 10 \text{ k}\Omega$ dan $R2 = 4,7 \text{ k}\Omega$ mereduksi tegangan motor 12V menjadi sekitar 3,87V, yang berada dalam rentang aman ADC Arduino (0-5V) dengan faktor skala 3,074 untuk konversi balik. Kombinasi ketiga sensor ini memungkinkan *monitoring* kondisi operasional motor secara komprehensif, yang dikirimkan ke Python GUI melalui komunikasi serial dengan format: sudut, RPM, dan PWM.

E. Analisa Sistem Monitoring Real-Time Berbasis PC

Arsitektur sistem *monitoring* yang dibangun dalam proyek ini memisahkan tanggung jawab komputasi secara hierarkis: Arduino Uno menjalankan *loop* kendali PID *real-time* pada laju 100 ms, sementara PC mengeksekusi deteksi gestur (MediaPipe), *rendering* GUI, dan visualisasi grafik. Pemisahan ini merupakan desain yang tepat mengingat keterbatasan memori dan kemampuan pemrosesan Arduino Uno (ATmega328P, 2 KB SRAM, 16 MHz). Menurut artikel KMTech (2025) yang membahas penggunaan kontroler PID dalam sistem kontrol otomatis industri, pemisahan antara lapisan kendali *real-time* dan lapisan *supervisory/monitoring* adalah praktik standar dalam sistem kendali industri modern.

Komunikasi serial dengan *baud rate* 115200 bps memberikan *bandwidth* yang cukup untuk transmisi data sensor (sudut, RPM, PWM) dengan frekuensi 10 Hz (setiap 100 ms), sekaligus penerimaan setpoint dari Python. Latensi komunikasi serial USB pada kecepatan ini secara tipikal berada di bawah 5 ms, yang dapat diabaikan dibandingkan dengan periode *sampling* sistem kendali (100 ms). Hal ini memastikan bahwa *delay* komunikasi tidak mengganggu stabilitas *loop* kendali PID yang berjalan di Arduino.

VI. KESIMPULAN

Berdasarkan hasil perancangan, implementasi, dan analisa, proyek ini berhasil membangun sistem kendali motor DC berbasis PID secara optimal menggunakan Arduino Uno sebagai kontroler utama. Implementasi algoritma PID diskrit dengan parameter $K_p = 3,30$, $K_i = 0,50$, dan $K_d = 0,002$ serta periode *sampling* 100 ms terbukti mampu memberikan respons kendali yang stabil terhadap berbagai nilai setpoint (35–80 RPM). Hal ini sejalan dengan teori kendali PID dari Ogata (2010) bahwa kombinasi aksi proporsional, integral, dan derivatif secara simultan menghasilkan performa yang optimal. Stabilitas dan keamanan sistem ini juga semakin diperkuat oleh implementasi fitur *anti-windup* integral pada rentang ± 255 serta mekanisme *reset* paksa saat setpoint bernilai nol. Fitur tersebut secara efektif mencegah akumulasi *error* yang berlebihan pada kondisi saturasi aktuator, sebagaimana direkomendasikan dalam literatur kendali motor DC (Jurnal Sains dan Teknologi, 2020; Jurnal ELKOMIKA, 2021).

Sebagai inovasi antarmuka, sistem deteksi gerakan tangan menggunakan MediaPipe Hands dan OpenCV berhasil diintegrasikan menjadi mekanisme input setpoint yang intuitif. Pengenalan jumlah jari di depan webcam secara langsung menentukan kecepatan target motor, membuktikan bahwa visi komputer dapat menjadi alternatif antarmuka elektromekanis yang efektif dan responsif (ResearchGate, 2024; ZBotic, n.d.; Antivirus, n.d.). Performa sistem turut ditunjang oleh sensor magnetik AS5600 beresolusi 12-bit (4096 posisi/putaran) dan *low-pass filter* digital yang memastikan pembacaan kecepatan tetap halus dan akurat. Dengan tambahan sensor arus ACS712-20A dan *voltage divider* untuk pengukuran tegangan, sistem mampu memantau kondisi operasional motor secara komprehensif dan *real-time* melalui GUI Python tanpa perlu mengandalkan infrastruktur Internet of Things (IoT).

Keberhasilan operasional alat ini tidak terlepas dari arsitektur sistem yang memisahkan lapisan kendali *real-time* (Arduino Uno) dari lapisan *monitoring* dan antarmuka pengguna (PC berbasis Python). Desain yang difasilitasi oleh komunikasi serial USB berkecepatan 115200 bps ini menghasilkan latensi sangat rendah (< 5 ms), sehingga stabilitas *loop* kendali PID tetap terjaga layaknya standar otomasi industri modern yang memisahkan lapisan kendali dan *supervisory* (KMTech, 2025). Secara keseluruhan, integrasi sistem kendali klasik, visi komputer, dan mikrokontroler terjangkau ini terbukti fungsional. Ke depannya, penelitian ini membuka peluang pengembangan lebih lanjut, seperti penerapan *tuning* otomatis menggunakan metode Cohen-Coon atau IMC berdasarkan identifikasi model FOPDT *open-loop*, penambahan kendali arah putaran motor, hingga transisi ke platform ESP32 untuk kapabilitas nirkabel.

VII. DAFTAR PUSTAKA

- Ogata, K. (2010). *Modern Control Engineering* (5th ed.). Prentice Hall.
- "Hand Gesture-Based Servo Motor Control Using Edge Computing". (2024). ResearchGate. https://www.researchgate.net/publication/391195487_Hand_Gesture-Based_Servo_Motor_Control_Using_Edge_Computing
- "Applications of DC Motor Controllers in Robotics". (n.d.). The Control Company. <https://www.thetacontrols.com/blogs/Applications-of-DC-Motor-Controllers-in-Robotics>
- "Gesture Recognition using Camera and MediaPipe on Raspberry Pi". (n.d.). ZBotic. <https://zbotic.in/gesture-recognition-using-camera-and-mediapipe-on-raspberry-pi/>
- Kontrol Proporsional Integral Derivatif (PID) pada Kecepatan Sudut Motor DC. (2021). Elkomika. <https://ejurnal.itenas.ac.id/index.php/elkomika/article/view/4303>
- Mengenal PID Controller dalam Sistem Kontrol Otomatis Industri. (2025). KMTech. <https://www.kmtech.id/post/mengenal-pid-controller-dalam-sistem-kontrol-otomatis-industri>
- Implementasi Kendali PID pada Kecepatan Motor DC. (2024). Jurnal Teknik Elektro dan Computer Engineering. <https://journal.ittelkom-pwt.ac.id/index.php/jtece/article/view/1461>
- Rancang Bangun Sistem Kendali Kecepatan Motor DC. (2020). Jurnal Sains dan Teknologi. <https://jurnal.poltekba.ac.id/index.php/jst/article/view/775>
- Kontrol Kecepatan Motor DC dengan Metode PID. (n.d.). Universitas Brawijaya. https://repository.ub.ac.id/141858/1/KONTROL_KECEPATAN_MOTOR_DC_DENGAN_METODE_PID_MENGGUNAKAN_VIS.pdf
- Kendali Kecepatan Motor DC Menggunakan Pengendali PID. (n.d.). STRING. <https://journal.lppmunindra.ac.id/index.php/STRING/article/view/13026>
- Implementasi kontrol PID untuk analisis pengaturan kecepatan motor DC. (2021). SNTEI. <http://jurnal.poliupg.ac.id/index.php/sntei/article/view/2802>
- Pengenalan Gerakan Tangan untuk Kontrol Slide Presentasi. (n.d.). JUPTI. <https://journalcenter.org/index.php/jupti/article/download/5332/4137>
- PENGENALAN GESTUR GERAKAN JARI UNTUK MENGONTROL. (n.d.). Antivirus. <https://ejournal.unisbablitar.ac.id/index.php/antivirus/article/download/2508/1375/7597>